

Project 2 — ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses

Title: ResearchHand Tool Agent Get Verified Answers Not Guesses

Type: Multi-agent (2 agents) · **Stack:** Python, LangChain (LCEL), an agent loop you build yourself ·

Level: Beginner–Intermediate **Tagline:** A tool-using Worker agent paired with a Verifier agent that checks the Worker's answer before it ever reaches the user.

Overview

ResearchHand is a tool-using agent that answers questions by actually looking things up — calling real tools instead of guessing from memory — and pairs that Worker with a second, independent Verifier agent that checks the answer before it ever reaches the user. It solves a real problem: a model working from memory alone states a wrong answer just as confidently as a right one, which is dangerous for anything where a wrong answer stated confidently is worse than no answer at all. Anyone building an assistant that needs live or exact information — a price lookup, a calculation, an internal API call — instead of a memorized guess would want something like this.

Features

- Hand-built think → act → observe agent loop (no `AgentExecutor`) that calls 3+ real, Pydantic-validated tools to answer a question
- Hard step limit that guarantees the agent can never loop forever on an ambiguous or tricky task
- Automatic recovery from a failing tool — errors are surfaced to the model as labeled results instead of causing a crash or a made-up answer
- Independent Verifier agent that reviews every Worker answer against the original question, with its own separate system prompt
- Full step-by-step logging of both agents, so any run can be checked and explained afterward
- Retry-with-feedback flow: a flagged answer can send the Worker back to redo the task using the Verifier's critique

Tech Stack

- Python
- LangChain (LCEL)

- OpenAI API
- Pydantic (tool argument validation)
- Hand-built agent loop (no `AgentExecutor`)

Architecture

A **Worker** agent runs a think → act → observe loop around 3+ Pydantic-validated tools, with a hard iteration limit, until it produces a final answer. That answer, together with the original question, is passed to a separate **Verifier** agent that has its own system prompt and never sees the Worker's step log — only the question and the final answer, so it can't be swayed by how hard the Worker "tried." This is a Generator → Critic pair: two independent agents with a real handoff, not one agent checking its own work.

Charter (what this project is)

Two agents: a Worker (3+ real tools, a hard limit on steps, recovers cleanly from a tool failure) and a Verifier (a second, separate check that reviews the Worker's final answer against the original question before it goes out). This is where "agent" stops being just a buzzword — you build the loop yourself before ever using `AgentExecutor` — and where you get your first taste of a creator-and-checker pair, the same pattern Project 4's Writer/Reviewer builds on.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Think about asking a smart friend a factual question. If they just guess from memory, they might sound confident and still be wrong. A friend you'd actually trust looks things up, does the math instead of estimating it, and — for anything important — has a second person check the answer before it goes to you.

That's exactly what you're building here. The **Worker** agent is that friend who looks things up: given a task, it can call real tools (a search, a calculator, a real API) instead of just guessing, and it keeps working step by step until it has a real answer — with a hard limit so it can never spin forever on a confusing task. The **Verifier** agent is the second person checking the work: it reads the original question and the Worker's answer side by side and asks "does this actually answer the question, and does it match what the tools really found?" — and if not, it flags it instead of letting a shaky answer slip through.

Put together, you get a small pipeline: a task comes in, the Worker researches and acts using real tools, and the Verifier double-checks the result before anyone sees it. That "generator, then an independent critic" shape is one of the most common, most useful patterns in real agent systems —

support bots, research tools, anything where a wrong answer stated confidently is worse than no answer at all.

The Steps below build this up gradually: Step 1 proves the underlying LLM chain works with no agent complexity at all. Step 2 adds exactly one tool and a hand-built loop — the smallest possible real agent. Step 3 grows that into a real Worker with several tools, a safety limit, and failure recovery. Step 4 adds the Verifier and completes the two-agent handoff.

What you're actually building, in one line: a Worker agent that calls real tools in a loop to answer a question, plus a separate Verifier agent that checks the Worker's answer before it goes out.

Why this needs to exist: a model working from memory alone states a wrong answer just as confidently as a right one — for tasks where a wrong answer stated confidently is worse than no answer, you need something that actually looks things up, and something else that double-checks the result.

When you'd reach for this at a real job: when an assistant needs to answer with live or exact information — a price, a stock lookup, a calculation — instead of a memorized guess, and a wrong answer would actually cost something.

How it works, mechanically: the Worker runs a think → act → observe loop with a hard step limit, calling tools as needed; once it stops, the Verifier reads the original question and the Worker's final answer side by side and flags it if the two don't actually match.

Why not just do it some simpler/different way: why not skip the loop and just give the model all the tool results in the prompt upfront? Because you often don't know in advance which tool, or which follow-up lookup, a given question will need — the loop lets the model decide one step at a time. Why not skip the Verifier and just trust the Worker's own answer? Because a model checking its own work is checking against the same blind spots that produced the mistake in the first place — a second, independent read of question vs. answer catches things self-review misses.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows a model that can *act*, not just talk — the difference between a chatbot and something that can look things up, calculate, or take a real action for you.
- **Why it matters:** this is the single most reused building block in AI work — supervisors, RAG agents, and every multi-agent pattern later are all built from this same basic piece, repeated. Someone reviewing your code sees a clean, hand-built agent loop — not just a one-line `AgentExecutor` call with no understanding behind it.
- **When you'd build something like this at a real job:** any assistant that needs to check live data, call internal services, or do multi-step lookups before answering — support bots, internal tools, research helpers.
- **How it's built:** a think → act → observe loop with a hard step limit, 3+ Pydantic-checked tools, and a step-by-step log — so every run can be checked afterward, not just a black-box final answer.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The agent loops forever on an unclear or tricky task	A hard <code>max_iterations</code> limit set in your code, which raises a clear error with the partial log attached — never trust the model to decide to stop on its own
The model ignores a tool's error and states a made-up result as fact	Return errors as clear, labeled tool results (not silent failures), so the model actually has what it needs to react correctly — and test this path directly
Two tools with overlapping jobs cause unreliable choices	Sharpen the tool descriptions until their jobs don't overlap — this is a prompt-writing fix, not a code fix

Setup (do this once, before Step 1)

```
cd project_2_researchhand
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langchain-core python-dotenv pydantic
pip freeze > requirements.txt
```

Same `.env` / `.env.example` setup as Project 1 (`OPENAI_API_KEY` , `LOG_LEVEL`) — copy it over if you're continuing from there, or make a new one if starting fresh here.

Built During These Documents

[05_langchain_fundamentals](#) → [06_tools_function_calling](#) → [07_ai_agents](#)

Plan Before You Code

See [15_five_projects_index](#): write out Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks before writing the agent loop.

How To Build This — Step by Step

Step 1 — A Single LCEL Chain That Answers One Question, No Tools

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 1 of 4: A Single LCEL Chain That Answers One Question, No Tools

What this step does: proves the LCEL chain works on its own, with a task simple enough that it needs no tool or loop — keeping "does my chain work" separate from "does my agent loop work" (that's Step 2's job). **Why this step matters:** every later step in this project reuses this exact chain — if it's broken, you'd be debugging two things at once (the chain and the loop) instead of one.

When you'll hit this for real: any time you're about to build a bigger agent and want the core "just answer" logic proven correct in isolation, before wrapping it in anything more complex. **Read first:** [05_langchain_fundamentals Core Concepts](#) — especially "LCEL: connecting pieces with `|`".

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build one plain LCEL chain: `prompt | ChatOpenAI | output_parser` — for a task with no tools at all (like "answer this research question from what the model already knows"). Pick a question the model can plausibly answer from training, not one needing live data — a stale or slightly-off answer here is expected, not a bug, since this step deliberately has no way to look anything up yet. 2. Test it directly — no loop, no tool, just `prompt → model → parsed answer`. This proves the chain works before any agent complexity gets added.

Your files after Step 1:

```
project_2_researchhand_tool_agent/
├── chain.py          → build_simple_chain() -> Runnable, no tools
└── main.py           → calls the chain directly, prints the answer
```

Step 2 — One Tool, One Loop: the Smallest Possible Working Agent

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 2 of 4: One Tool, One Loop: the Smallest Possible Working Agent

What this step does: proves the think → act → observe loop actually works, with exactly one tool — the smallest possible thing you could really call "an agent." Keeps "does my loop work" separate from "do all my tools work" (Step 3's job). **Why this step matters:** a hand-built loop with one tool is small enough to fully understand line by line — that understanding is what makes Step 3's multi-tool loop (and every `AgentExecutor` you use afterward) clear instead of magic. **What's new vs. Step 1:** one tool gets added; a loop you build by hand wraps Step 1's chain. **What stays the same:** Step 1's chain is still what generates the model's replies inside the loop — you're wrapping it with agent behavior, not throwing it away. **When you'll hit this for real:** this is the exact moment "chatbot" becomes "agent" in any project — the first time you give a model the ability to act, not just answer. **Read first:** [06_tools_function_calling Core Concepts](#), [07_ai_agents Core Concepts](#) — "The think → act → observe loop".

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Define exactly one tool with a Pydantic argument shape (see Doc06) — pick the simplest, pure-logic one. 2. Build the loop yourself (think → act → observe → repeat) around Step

1's chain, now with that one tool added — **do this before** reaching for `AgentExecutor`, as Doc07 explains. There's deliberately no iteration limit yet (that's Step 3), but watch for the model calling the tool repeatedly with near-identical input — it usually means the tool's result isn't being fed back into the message history clearly enough for the model to recognize it already has an answer. 3. Test it: a prompt that clearly needs the tool, and one that doesn't. Confirm the loop only calls the tool when it actually should.

Your files after Step 2:

```
project_2_researchhand_tool_agent/
├─ chain.py
├─ tools.py           → 1 @tool-decorated function, Pydantic arguments
├─ agent.py          → run_agent(task) -> AgentResult (your own loop, no
└─ main.py           → runs the loop, prints each think/act/observe step
```

Step 3 — Scaling to 3+ Tools With a Hard Step Limit and Failure Recovery

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 3 of 4: Scaling to 3+ Tools With a Hard Step Limit and Failure Recovery

What this step does: turns Step 2's small loop into the real Worker — more tools to pick from, a hard ceiling so it can never run forever, and a clean recovery when a tool actually fails. **Why this step matters:** a demo loop with one reliable tool never shows you what breaks in production — real tools fail, and real tasks are ambiguous about which tool to use; this step is where the agent has to survive both on purpose. **What's new vs. Step 2:** 2+ more tools (including one that sometimes fails) get added; a `max_iterations` limit and step-by-step logging get added. **What stays the same:** the loop itself from Step 2 — you're giving it more tools and a safety limit, not rewriting how it thinks. **When you'll hit this for real:** the moment any agent you've built moves from "demo" to "something I'd actually deploy" — a step limit and failure recovery are non-negotiable the instant real, unpredictable users are involved. **Read first:** [07_ai_agents Core Concepts — "Setting a limit on the number of steps"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add 2+ more tools with Pydantic shapes — at least one backed by a real service (reuse Doc02's client), one that's set up to sometimes fail on purpose. Write each tool's description to state clearly when *not* to use it, not just what it does — once two tools' jobs could plausibly overlap, the model is choosing based entirely on the description text, not the function name. 2. Add the hard `max_iterations` limit, raising a clear `MaxIterationsExceeded` error with the partial log attached when it's hit. 3. Add step-by-step logging for every think/act/observe round. 4. Test: an unclear prompt (two tools could apply), a tool-failure case (confirm it recovers, doesn't crash), and a tricky prompt built to hit the step limit.

Your files after Step 3:

```

project_2_researchhand_tool_agent/
├─ chain.py
├─ tools.py           → 3+ @tool-decorated functions, Pydantic arguments
├─ agent.py          → run_agent(task, max_iterations) -> AgentResult, li
└─ main.py           → runs the Worker agent, prints the log + final ans

```

Step 4 — A Verifier Agent That Catches the Worker's Bad Answers (final: 2 agents)

Project: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — Step 4 of 4: A Verifier Agent That Catches the Worker's Bad Answers

What this step does: adds a second, separate agent whose only job is judging the first agent's answer — your first real multi-agent handoff with actual value, not just routing between two functions. **Why this step matters:** a single agent can be confidently wrong with no one checking it — pairing it with an independent Verifier is the cheapest, most reusable safety net in agent design, and it's the same generator/critic shape you'll see again in Project 4 and beyond. **What's new vs. Step 3:** a new Verifier agent, plus a check-then-maybe-retry flow around the Worker's final answer. **What stays the same:** the Worker agent itself — Step 4 doesn't change its loop, tools, or logic at all; the Verifier just watches it from the outside. **When you'll hit this for real:** any high-stakes agent answer (financial, medical, legal-adjacent, or just "customer-facing and embarrassing if wrong") where a second, independent check before the user sees it is worth the extra cost. **Read first:** [11_multi_agent_systems Core Concepts — "Generator → Critic → Revision"](#) (you're building a small version of this pattern now, well before Doc11's full LangGraph version).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write a **Verifier agent**: a separate model call given the original question and the Worker's final answer, whose only job is to judge — does this answer actually address the question, and does it match what the tools actually returned? Give it its own separate system prompt (it should *not* share the Worker's personality). Show it only the question and the final answer, not the Worker's full step log — a Verifier that can see how hard the Worker "tried" tends to rubber-stamp effort instead of judging the actual answer, the same way a grader shouldn't be swayed by how long a student spent on an exam. 2. Connect the main flow: Worker runs to the end → Verifier checks the result → if the Verifier flags a problem, either run the Worker again with the Verifier's feedback added, or show the flag to the user instead of quietly returning a possibly-wrong answer. 3. Log both agents' output separately (Worker's log, Verifier's verdict), so anyone reviewing it can see the handoff, not just the final answer. 4. Test it: on purpose, give the Worker a task likely to produce a shaky answer (like a tool returning unclear data), and confirm the Verifier actually catches it at least once in your test set. A Verifier that always approves isn't really tested — it's just for show.

Your files after Step 4 (final):

```

project_2_researchhand_tool_agent/
├─ chain.py

```

```

├─ tools.py
├─ agents/
│   └─ worker_agent.py      → run_worker(task, max_iterations) -> AgentResult
│   └─ verifier_agent.py    → verify(question, worker_result) -> VerifierVerdi
├─ main.py                  → Worker runs → Verifier checks → retry or show t
└─ test_agent.py

```

Final Deliverable: ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses — a 2-agent system. A Worker solves multi-step, tool-using tasks with a hard safety limit. An independent Verifier checks every answer before it reaches the user.

Why this really is multi-agent (said plainly): this is a simple **Generator** → **Critic** pair (see Doc11) — no shared graph state yet (that's Project 3), no supervisor routing between more than two roles (that's Project 4). But it genuinely is two independent agents with a real handoff, and a real chance for the Verifier to catch something.

Checklist Before You Call This Done

- [] 3+ tools registered on the Worker, each with a precise, testable description
- [] A hard step limit on the Worker — never loops forever, even on a tricky input
- [] At least one tool actually fails mid-run in a real test, and the Worker recovers instead of crashing or making things up
- [] The Verifier agent runs independently on every Worker answer, with its own separate prompt
- [] At least one test case where the Verifier actually flags a bad Worker answer (not just always approving)
- [] Full step-by-step log for every run, for both agents
- [] You can explain, for the unclear test prompts, why the Worker chose the tool it did, and why the Verifier approved or rejected

Full requirements, test cases, and hints: [07_ai_agents/README.md](#).

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**

Part of a 13-project multi-agent AI engineering curriculum. See the [full curriculum](#) for the complete learning path and all other projects.